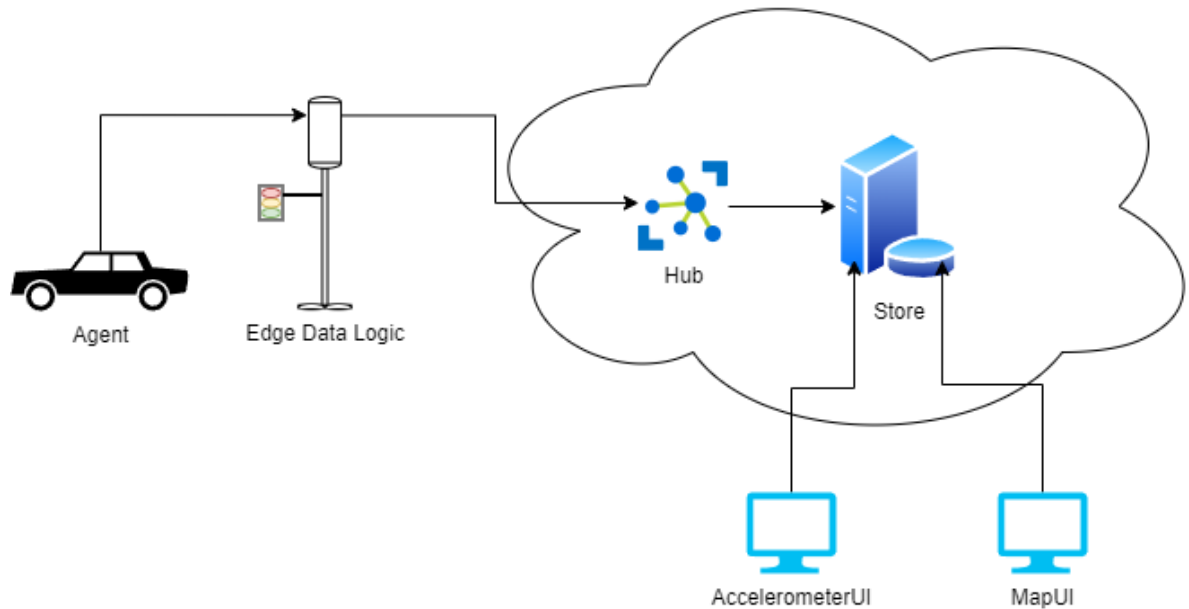


## Лабораторна робота №2

### Опис проекту



Завдання – розробити комплексну систему моніторингу та аналізу стану дорожнього покриття. Система може мати можливість збирати дані про різні параметри, такі як нерівності, тріщини, вибоїни та інші дефекти, які можуть вплинути на якість дороги. Зібрані дані слід проаналізувати, щоб визначити закономірності та тенденції, які можуть допомогти у прогнозуванні потреб у технічному обслуговуванні дорожньої мережі. Система також може мати можливість генерувати звіти та сповіщення для відповідних органів влади та/або водіїв.

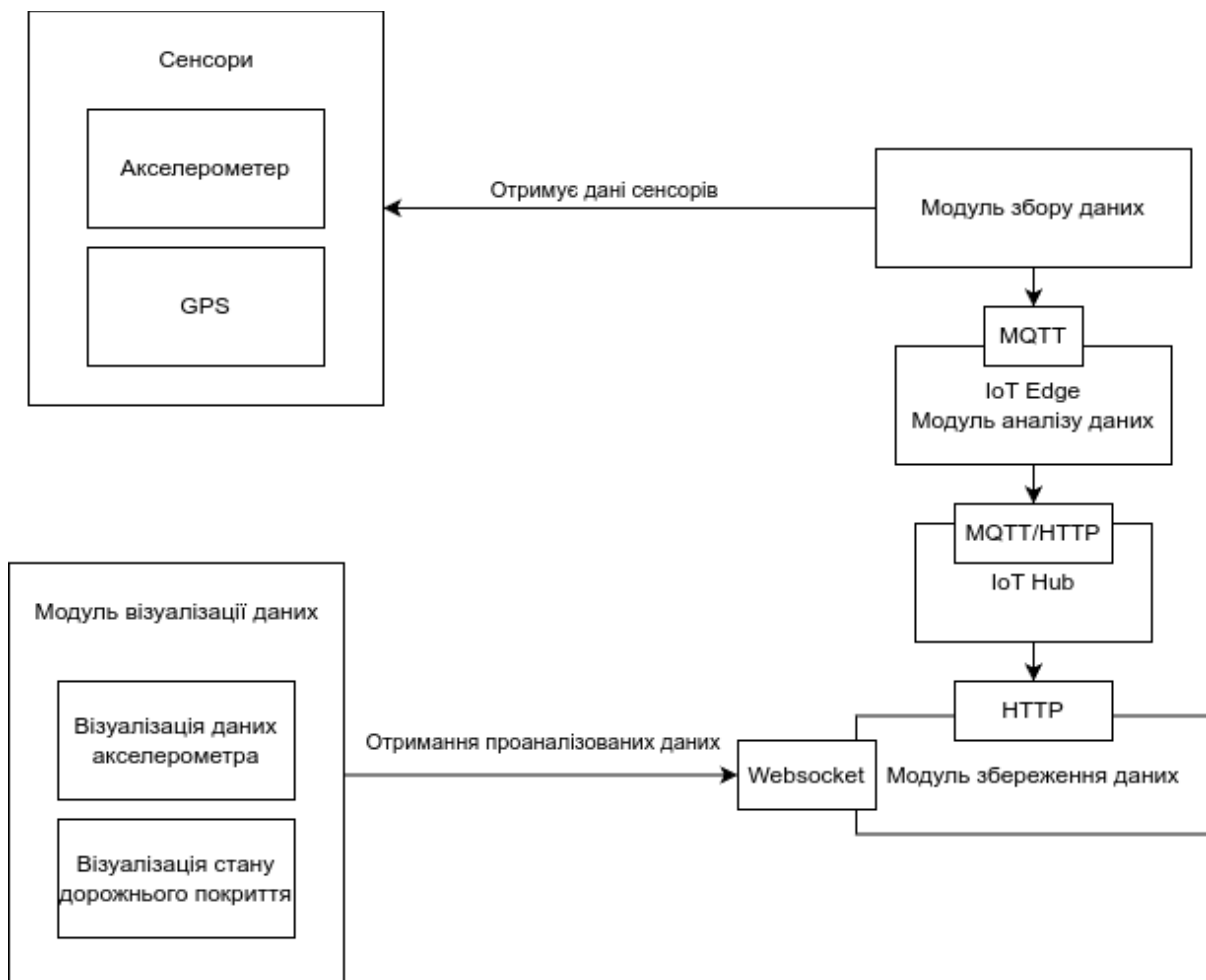
Метою цієї системи є підвищення безпеки дорожнього руху, зниження витрат на технічне обслуговування та покращення загального досвіду водіння для автомобілістів.

В систему входять пристрій на машині (stm32), edge пристрій, хмарні або on-prem сервіси.

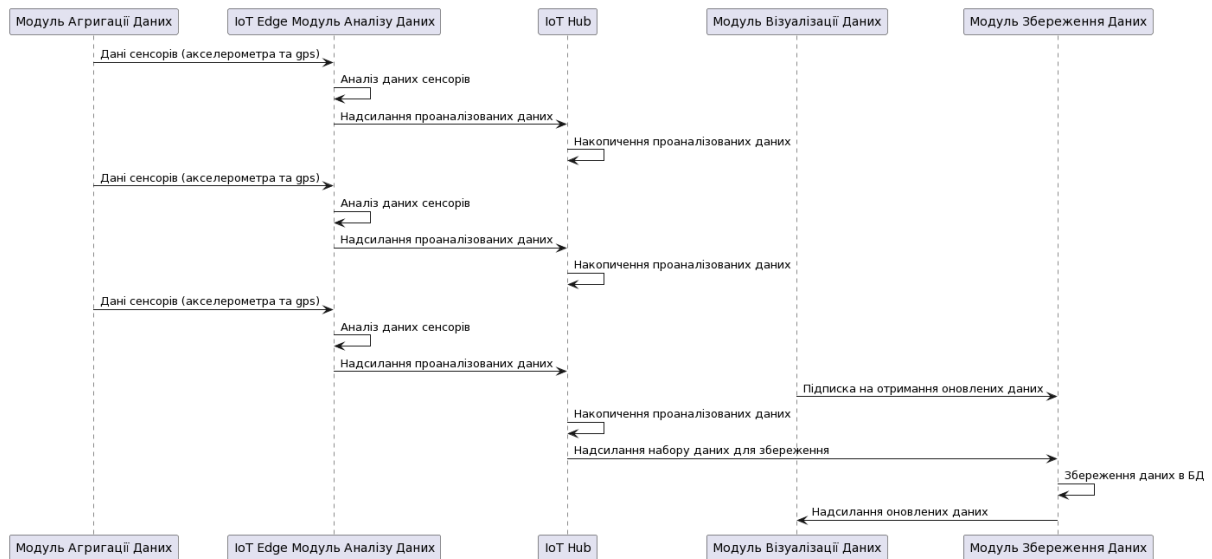
Система містить наступні частини (дозволяється розширювати):

- Модуль збору даних (Agent) - програма агент на машині, яка знімає показники сенсорів (наприклад раз в 100мс) та відправляє їх на сервер.

- Модуль первинної обробки даних (Edge Data Logic) - програма яка займається первинною обробкою даних, в нашому випадку визначає стан дорожнього покриття та відправляє проаналізовані дані на Hub.
- Модуль накопичення даних (Hub) - сервіс який займається накопиченням та аналізом (preprocessing) отриманих даних від Agent та подальшим зберіганням оброблених даних в БД. Перед збереженням дани потрібно накопичити (наприклад 10 записів), так як такі дані простіше аналізувати і запис в БД множини даних буде швидшим.
- Store - арі для отримання та збереження даних в БД.
- Модуль візуалізації (Accelerometer UI, Map UI) - інтерфейс візуалізації (моніторингу) результатів системи.



Архітектура системи



## Юзкейс роботи

### Постановка задачі

Для зберігання та доступу до даних потрібно реалізувати Store apі який буде зберігати проаналізовані дані в базу даних. Також потрібно реалізувати спосіб отримання нових даних для UI клієнтів.

### Кроки виконання

Надалі буде показана реалізація на python, проте ніхто не забороняє використовувати той технологічний стек який для вас є оптимальним.

До початку роботи потрібно встановити [Docker](#).

1. Створення віртуального середовища. Пакет **virtualenv** повинен бути встановлений в системі.

```
python -m venv venv
```

2. Активація середовища

```
source ./venv/bin/activate # linux
```

```
.\venv\Scripts\activate.bat # windows
```

### 3. Встановимо необхідні залежності(бібліотеки)

```
pip install fastapi  
pip install "uvicorn[standard]"  
pip install SQLAlchemy  
pip install psycopg2  
pip install pydantic
```

### 4. Збережемо залежності в requirements.txt

```
pip freeze > requirements.txt
```

### 5. Структура проєкту

```
|— config.py  
|— docker  
|   |— db  
|   |   |— structure.sql  
|   |— docker-compose.yaml  
|— Dockerfile  
|— main.py  
|— README.md  
|— requirements.txt
```

### 6. Створемо файл конфігурацій

```

1  import os
2
3
4  def try_parse(type, value: str):
5      try:
6          return type(value)
7      except Exception:
8          return None
9
10 # Configuration for POSTGRES
11 POSTGRES_HOST = os.environ.get("POSTGRES_HOST") or "localhost"
12 POSTGRES_PORT = try_parse(int, os.environ.get("POSTGRES_PORT")) or
13 5432
14 POSTGRES_USER = os.environ.get("POSTGRES_USER") or "user"
15 POSTGRES_PASSWORD = os.environ.get("POSTGRES_PASS") or "pass"
16 POSTGRES_DB = os.environ.get("POSTGRES_DB") or "test_db"

```

7. Наступним кроком реалізуємо таблиці для бази даних та моделі для роботи з ними.

main.py

```

1  # SQLAlchemy setup
2  DATABASE_URL =
3  f"postgresql+psycopg2://{POSTGRES_USER}:{POSTGRES_PASSWORD}@{POSTGRES_HOST}:{POSTGRES_PORT}/{POSTGRES_DB}"
4  engine = create_engine(DATABASE_URL)
5  metadata = MetaData()
6  # Define the ProcessedAgentData table
7  processed_agent_data = Table(
8      "processed_agent_data",
9      metadata,
10     Column("id", Integer, primary_key=True, index=True),
11     Column("road_state", String),
12     Column("x", Float),
13     Column("y", Float),
14     Column("z", Float),
15     Column("latitude", Float),
16     Column("longitude", Float),
17     Column("timestamp", DateTime),
18 )

```

8. Реалізуємо моделі даних.

```
1 # FastAPI models
2 class AccelerometerData(BaseModel):
3     x: float
4     y: float
5     z: float
6
7
8 class GpsData(BaseModel):
9     latitude: float
10    longitude: float
11
12
13 class AgentData(BaseModel):
14     accelerometer: AccelerometerData
15     gps: GpsData
16     timestamp: datetime
17
18     @classmethod
19     @field_validator('timestamp', mode='before')
20     def check_timestamp(cls, value):
21         if isinstance(value, datetime):
22             return value
23         try:
24             return datetime.fromisoformat(value)
25         except (TypeError, ValueError):
26             raise ValueError(
27                 "Invalid timestamp format. Expected ISO 8601 format
28 (YYYY-MM-DDTHH:MM:SSZ).")
29
30
31 class ProcessedAgentData(BaseModel):
32     road_state: str
33     agent_data: AgentData
34
35 # Database model
36 class ProcessedAgentDataInDB(BaseModel):
37     id: int
38     road_state: str
39     x: float
40     y: float
41     z: float
```

```
42     latitude: float
43     longitude: float
44     timestamp: datetime
45
```

9. Реалізуємо логіку підписки на оwoлення даних. Це буде використовуватись для оновлення даних на UI.

```

1  # FastAPI app setup
2  app = FastAPI()
3
4  # WebSocket subscriptions
5  subscriptions: Set[WebSocket] = set()
6
7
8  # FastAPI WebSocket endpoint
9  @app.websocket("/ws/")
10 async def websocket_endpoint(websocket: WebSocket):
11     await websocket.accept()
12     subscriptions.add(websocket)
13     try:
14         while True:
15             await websocket.receive_text()
16     except WebSocketDisconnect:
17         subscriptions.remove(websocket)
18
19
20 # Function to send data to subscribed users
21 async def send_data_to_subscribers(data):
22     for websocket in subscriptions:
23         await websocket.send_json(json.dumps(data))

```

10. Реалізуємо CRUD операції для ProcessedAgentData in main.py.



```

1  # FastAPI CRUDL endpoints
2
3  @app.post("/processed_agent_data/")
4  async def create_processed_agent_data(data:
5  List[ProcessedAgentData]):
6      # Insert data to database
7      # Send data to subscribers
8
9  @app.get(
10     "/processed_agent_data/{processed_agent_data_id}",
11     response_model=ProcessedAgentDataInDB
12 )
13 def read_processed_agent_data(processed_agent_data_id: int):
14     # Get data by id
15
16
17 @app.get(
18     "/processed_agent_data/",
19     response_model=list[ProcessedAgentDataInDB]
20 )
21 def list_processed_agent_data():
22     # Get list of data
23
24
25 @app.put(
26     "/processed_agent_data/{processed_agent_data_id}",
27     response_model=ProcessedAgentDataInDB
28 )
29 def update_processed_agent_data(
30     processed_agent_data_id: int,
31     data: ProcessedAgentData
32 ):
33     # Update data
34
35 @app.delete(
36     "/processed_agent_data/{processed_agent_data_id}",
37     response_model=ProcessedAgentDataInDB
38 )
39 def delete_processed_agent_data(processed_agent_data_id: int):
40     # Delete by id

```

## 11. Занык FastApi

```

1 if __name__ == "__main__":
2     import uvicorn
3
4     uvicorn.run(app, host="127.0.0.1", port=8000)

```

12.Добавимо Dockerfile для запуску в docker.

Dockerfile

```

1 # Use the official Python image as the base image
2 FROM python:latest
3 # Set the working directory inside the container
4 WORKDIR /app
5 # Copy the requirements.txt file and install dependencies
6 COPY requirements.txt .
7 RUN pip install --no-cache-dir -r requirements.txt
8 # Copy the entire application into the container
9 COPY . .
10 # Run the main.py script inside the container when it starts
11 CMD ["uvicorn", "main:app", "--host", "0.0.0.0"]

```

13.Налаштуємо docker-compose.yaml для локального запуску та тестування програми.

structure.sql

```

1 CREATE TABLE processed_agent_data (
2     id SERIAL PRIMARY KEY,
3     road_state VARCHAR(255) NOT NULL,
4     x FLOAT,
5     y FLOAT,
6     z FLOAT,
7     latitude FLOAT,
8     longitude FLOAT,
9     timestamp TIMESTAMP
10 );

```

docker-compose.yaml

```
1  version: "3.9"
2  name: "road_vision__database"
3  services:
4    postgres_db:
5      image: postgres:latest
6      container_name: postgres_db
7      restart: always
8      environment:
9        POSTGRES_USER: user
10       POSTGRES_PASSWORD: pass
11       POSTGRES_DB: test_db
12     volumes:
13       - postgres_data:/var/lib/postgresql/data
14       - ./db/structure.sql:/docker-entrypoint-initdb.d/structure.sql
15     ports:
16       - "5432:5432"
17     networks:
18       db_network:
19
20
21   pgadmin:
22     container_name: pgadmin4
23     image: dpage/pgadmin4
24     restart: always
25     environment:
26       PGADMIN_DEFAULT_EMAIL: admin@admin.com
27       PGADMIN_DEFAULT_PASSWORD: root
28     volumes:
29       - pgadmin-data:/var/lib/pgadmin
30     ports:
31       - "5050:80"
32     networks:
33       db_network:
34
35
36   store:
37     container_name: store
38     build: ..
39     depends_on:
40       - postgres_db
41     restart: always
42     environment:
43       POSTGRES_USER: user
44       POSTGRES_PASSWORD: pass
45       POSTGRES_DB: test_db
46       POSTGRES_HOST: postgres_db
47       POSTGRES_PORT: 5432
48     ports:
49       - "8000:8000"
50     networks:
51       db_network:
```

52

53

54 networks:

55 db\_network:

56

57

58 volumes:

59 postgres\_data:

60 pgadmin-data:

## 14. Запуск програму в docker

```
cd docker
```

```
docker-compose up --build
```

## 15. Перевірка працездатності в Swagger (127.0.0.1:8000/docs)

**FastAPI** 0.1.0 0AS 3.1  
/openapi.json

**default** ^

|        |                                                 |                             |   |
|--------|-------------------------------------------------|-----------------------------|---|
| GET    | /processed_agent_data/                          | List Processed Agent Data   | ▼ |
| POST   | /processed_agent_data/                          | Create Processed Agent Data | ▼ |
| GET    | /processed_agent_data/{processed_agent_data_id} | Read Processed Agent Data   | ▼ |
| PUT    | /processed_agent_data/{processed_agent_data_id} | Update Processed Agent Data | ▼ |
| DELETE | /processed_agent_data/{processed_agent_data_id} | Delete Processed Agent Data | ▼ |

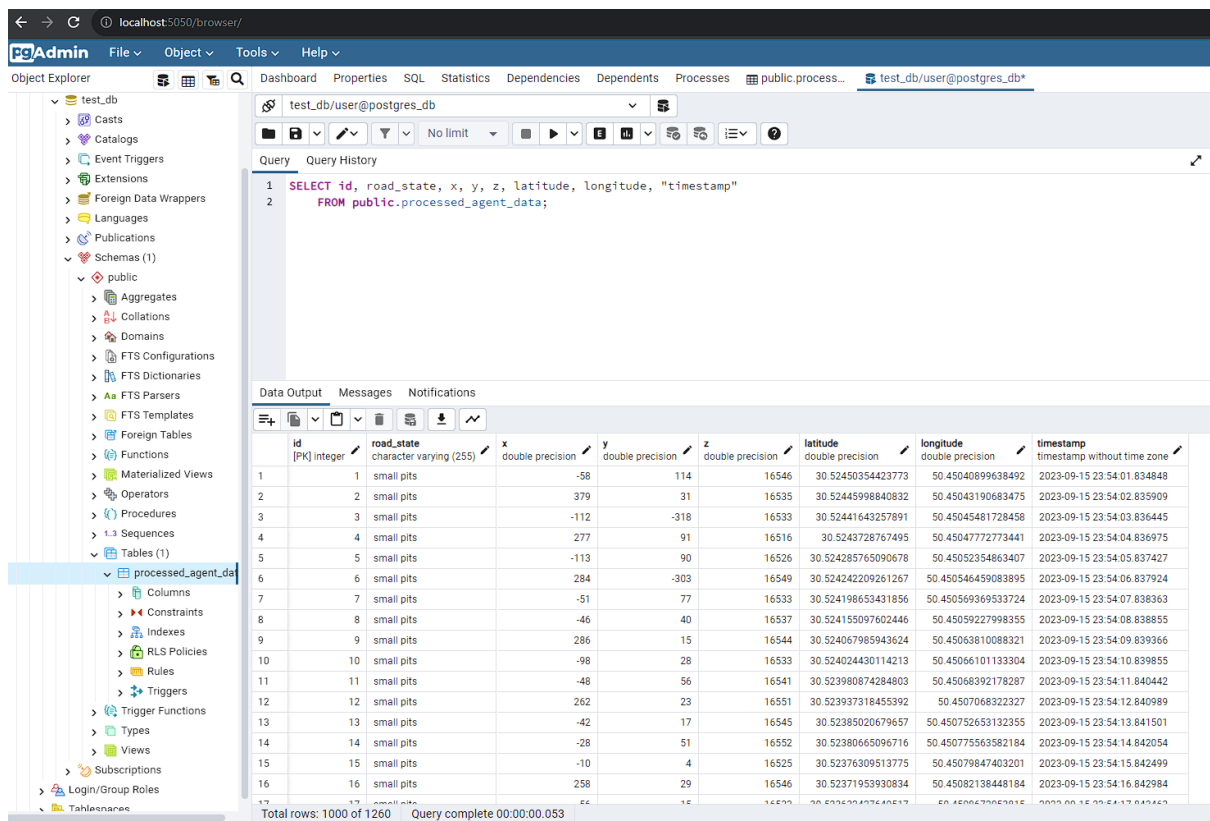
**Schemas** ^

**AccelerometerData** > Expand all object

**AgentData** > Expand all object

**GpsData** > Expand all object

## 16. Перевірка працездатності в PgAdmin (127.0.0.1:5050)



## User story for testing

| Дія                                     | Очікуваний результат                                            |
|-----------------------------------------|-----------------------------------------------------------------|
| Добавити ProcessedAgentData (1)         | В таблицю добавлений відповідний запис                          |
| Добавити ProcessedAgentData (2)         | В таблицю добавлений відповідний запис                          |
| Добавити ProcessedAgentData (3)         | В таблицю добавлений відповідний запис                          |
| Отримати список всіх ProcessedAgentData | Отримано 3 ProcessedAgentData а саме ProcessedAgentData (1,2,3) |
| Видалити ProcessedAgentData (2)         | 3 таблиці видалено відповідний запис                            |
| Отримати список всіх ProcessedAgentData | Отримано 2 ProcessedAgentData а саме ProcessedAgentData (1,3)   |
| Оновити ProcessedAgentData (3)          | В таблиці оновлено відповідний запис                            |

## **Очікувані результати в протоколі**

1. Посилання на git репозиторій з проектом
2. Реалізація CRUD операцій
3. Показ результатів роботи в Swagger та PgAdmin
4. Висновки